Projet MLSF – Un solveur SAT pour Meowdoku

Partie I

Pablo Bosch Cécile Chevrière

Octobre 2026

Organisation du rendu. Les programmes se trouvent dans le dossier `Session de travail` : le module `meowdoku.py` (lecture des grilles, encodage CNF, écriture DIMACS, vérification), le module `generateur.py` (génération de grilles) et le notebook `projet_partie1.ipynb`, qui exécute les programmes dans l'ordre des questions. Les grilles et les fichiers DIMACS produits sont rangés dans `grilles/` et `cnf/`.

1 Question 1 – Modélisation de Meowdoku

1.1 Variables

Pour une grille $N \times M$ découpée en régions $R_1, \dots, R_K$, on introduit une variable $x_{i,j}$ par case $(i,j) \in \{1,\dots,N\} \times \{1,\dots,M\}$, interprétée par « un chat est placé sur la case (i,j) ». Aucune autre variable n'est introduite. Dans le fichier DIMACS, $x_{i,j}$ porte le numéro $(i-1)M + j$, de 1 à NM.

1.2 Clauses

Chaque règle du jeu est traduite par une famille de clauses. « Au plus un chat » dans un ensemble de cases est exprimé en interdisant chaque paire de chats ; « au moins un chat » par une clause unique.

(1) au plus un chat par ligne : $\displaystyle\bigwedge_{i=1}^{N} \bigwedge_{1 \le j < j' \le M} (\neg x_{i,j} \vee \neg x_{i,j'})$

(2) au plus un chat par colonne : $\displaystyle\bigwedge_{j=1}^{M} \bigwedge_{1 \le i < i' \le N} (\neg x_{i,j} \vee \neg x_{i',j})$

(3) exactement un chat par région : $\displaystyle\bigwedge_{k=1}^{K} \Big(\bigvee_{(i,j) \in R_k} x_{i,j} \Big) \wedge \bigwedge_{k=1}^{K} \bigwedge_{\substack{c,c' \in R_k \\ c < c'}} (\neg x_c \vee \neg x_{c'})$

(4) pas de chats en diagonale : $\displaystyle\bigwedge_{i=1}^{N-1} \bigwedge_{j=1}^{M} \bigwedge_{\substack{j' \in \{j-1, j+1\} \\ 1 \le j' \le M}} (\neg x_{i,j} \vee \neg x_{i+1,j'})$

Deux cases voisines au sens du 8-voisinage sont soit sur la même ligne, soit sur la même colonne, soit en diagonale : les voisinages horizontal et vertical étant déjà interdits par (1) et (2), la famille (4) se limite aux diagonales. Les clauses identiques produites par plusieurs familles (par exemple deux cases d'une même région situées sur une même ligne) ne sont écrites qu'une fois.

Correction. À une valuation v on associe la configuration $S_v = \{(i,j) \mid v(x_{i,j}) = 1\}$. Chaque famille est exactement la traduction d'une règle : v satisfait (1) (resp. (2)) si et seulement si S_v contient au plus un chat par ligne (resp. colonne) ; v satisfait (3) si et seulement si chaque région contient au moins et au plus un chat de S_v ; enfin, compte tenu de (1) et (2), v satisfait (4) si et seulement si aucun couple de chats n'est adjacent. Ainsi $v \models F$ si et seulement si S_v est une solution, et $v \mapsto S_v$ est une bijection entre les modèles de F et les solutions de la grille : la formule est satisfiable si et seulement si la grille a une solution, et toute valuation témoin donne une solution.

Taille de la formule. Avant suppression des doublons, le nombre de clauses est

$$N\binom{M}{2} + M\binom{N}{2} + K + \sum_{k=1}^{K} \binom{|R_k|}{2} + 2(N-1)(M-1),$$

soit $O\big(NM(N+M)\big)$ en ordre de grandeur. Le tableau 1 donne les tailles obtenues pour les grilles générées (section 1.4).

$N = M = K$	4	6	8	10	15	20
variables	16	36	64	100	225	400
clauses	74	258	638	1 253	4 341	10 547

Table 1: Taille des formules CNF (nombre moyen de clauses sur 3 grilles générées par taille).

1.3 Implémentation et validation

La fonction `lire_grille` lit un fichier au format imposé et vérifie que les régions forment une partition de la grille (régions déclarées, non vides). `grille_vers_cnf` construit la liste des clauses, `ecrire_dimacs` écrit le fichier, et `convertir` enchaîne les trois (aussi utilisable en ligne de commande : `python meowdoku.py grille.txt sortie.cnf`). Pour la grille de la figure 1 de l'énoncé, on obtient 36 variables et 309 clauses.

La fonction `verifier_solution` contrôle directement les quatre règles du jeu sur une configuration de chats, indépendamment de la CNF. Elle a servi à valider l'encodage :

- la solution de la figure 2 de l'énoncé est licite et satisfait la CNF ;
- sur cinq grilles 4×4, pour chacune des 2^{16} valuations, « la valuation satisfait la CNF » et « la configuration associée est licite » coïncident ;
- sur des grilles 6×6 et 7×7, où l'énumération complète est hors de portée, il y a également accord sur toutes les permutations (un chat par ligne et par colonne) et sur 20 000 valuations aléatoires.

1.4 Génération de grilles

On se restreint au cas $N = M = K$. Pour garantir l'existence d'une solution, on construit *d'abord* une solution, *puis* les régions autour d'elle.

Étape 1 : placement des chats. Avec $N = M = K$, une solution place exactement un chat par ligne et par colonne : c'est une permutation σ (chat en $(i, \sigma(i))$). Les chats de deux lignes consécutives ne doivent pas être adjacents, d'où la contrainte $|\sigma(i+1) - \sigma(i)| \geq 2$. On place les chats ligne par ligne : pour chaque ligne, on essaie les colonnes libres compatibles avec le chat de la ligne précédente, dans un ordre aléatoire ; si aucune ne convient, on revient sur la

ligne précédente pour y essayer la colonne suivante. Le procédé trouve un placement dès qu'il en existe un. Il n'en existe pas pour $N = 2$ (deux chats d'une grille 2×2 sont toujours adjacents) ni pour $N = 3$ (le chat de la ligne 2 devrait être en colonne 1 ou 3, et ceux des lignes 1 et 3 tous deux dans la colonne opposée) ; le générateur s'utilise donc pour $N = 1$ ou $N \geq 4$.

Étape 2 : croissance des régions. Chaque chat est le germe d'une région. Tant qu'il reste des cases libres, on choisit une région au hasard et on lui ajoute une case libre voisine (haut, bas, gauche ou droite) de l'une de ses cases. Comme la grille est connexe, toutes les cases finissent par être attribuées.

Garantie. Chaque case appartient à exactement une région, et chaque région est connexe et contient exactement un chat, son germe. Les régions forment donc une partition de la grille en $K = N$ régions non vides, et le placement de l'étape 1 respecte les quatre règles : la grille obtenue possède au moins une solution. Cette solution est écrite en commentaire dans le fichier de la grille et une graine rend chaque génération reproductible (`python generateur.py N graine sortie.txt`). Pour les 51 grilles générées ($N = 4, \ldots, 20$, trois graines par taille), `verifier_solution` confirme que la solution construite est licite.

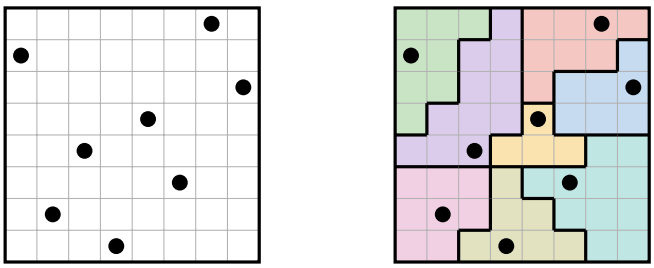


Figure 1: Génération d'une grille 8×8 : placement des chats (gauche), puis croissance des régions autour de chaque chat (droite).

Usage des assistants IA

Un assistant IA (Claude) a été utilisé pour proposer la démarche et une première version du code et du rapport, que nous avons relus, testés et repris.